PCSX-Redux

None

None

None

Table of contents

1. Home 4
2. PCSX-Redux menus 5
   - 2.1 File 6
   - 2.2 Emulation 7
   - 2.3 Configuration 7
   - 2.4 Debug 8
   - 2.5 Help 8
   - 2.6 GPU information 8
3. Compiling PCSX-Redux 10
   - 3.1 Getting the sources 10
   - 3.2 Windows 10
   - 3.3 Linux 11
   - 3.4 Compiling PSX code 12
4. Command Line Flags 14
5. Debugging 18
   - 5.1 Debugging with PCSX-Redux 18
   - 5.2 GDB server 19
   - 5.3 Connecting Ghidra to PCSX-Redux 28
   - 5.4 Misc Features 31
   - 5.5 VRAM viewer 34
   - 5.6 GPU Logger 35
6. Mips API 37
   - 6.1 Description 37
   - 6.2 Functions 38
7. Web server 40
   - 7.1 Activation 40
   - 7.2 REST API 40
8. Lua 43
   - 8.1 Introduction 43
   - 8.2 Loaded libraries 45
   - 8.3 Redux basic API 47
   - 8.4 Rendering 56
   - 8.5 File API 66
   - 8.6 Webserver Lua API 75
   - 8.7 Memory and registers 76

8.8 Events 79
8.9 Breakpoints 83
8.10 Inline assembler 86
8.11 Handling of PSX binaries 87
8.12 Case studies 91
9. Openbios 0
9.1 Purposes of Openbios 0
9.2 Building 0
9.3 Status 0
9.4 Organization 0
9.5 Technicalities 0
9.6 Commentary 0
9.7 Legality 0

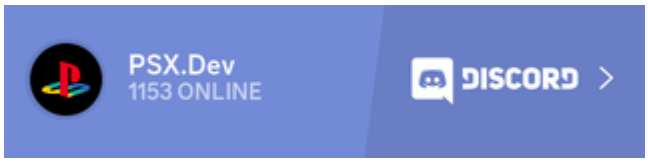
1. Home

Welcome to the PCSX-Redux emulator documentation.

You can get the emulator for various platforms here: https://github.com/grumpycoders/pcsx-redux#where

To discuss this emulator specifically, please join our Discord server:

To discuss PlayStation 1 development, hacking, and reverse engineering in general, please join the PSX.Dev Discord server:



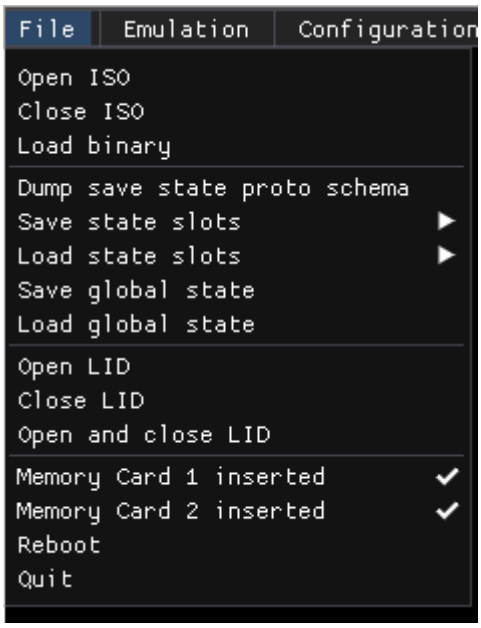
Compiling PCSX-Redux
Menus
Command line arguments
Debugging with PCSX-Redux
Internal MIPS api
Web Server
Lua API
OpenBios

2. PCSX-Redux menus

The menu bar holds some informations :

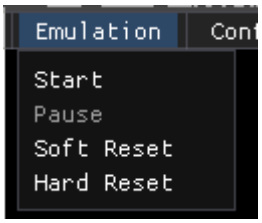
- CPU mode
- Game ID
- ImGui FPS counter (not psx internal fps)

2.1 File



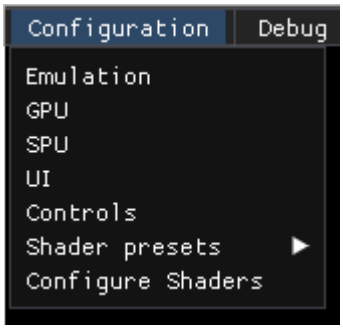
- Open ISO
- Close ISO
- Load Binary
- Dump save state proto schema
- Save state slots
- Load state slots
- Save global state
- Load global state
- Open Lid : Simulate open lid
- Close Lid : Simulate closed lid
- Open and Close Lid : Simulate opening then closing the lid
- MC1 inserted: Insert or remove Memory Card 1
- MC2 inserted: Insert or remove Memory Card 2
- Reboot : Restart emulator
- Quit

2.2 Emulation



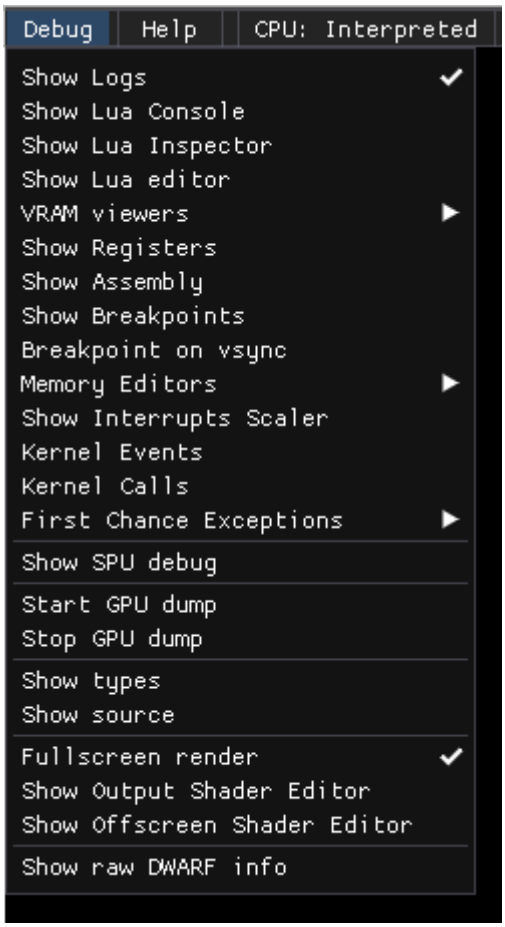
- Start (F5): Start execution
- Pause (F6): Pause execution
- Soft reset (F8): Calls Redux's CPU reset function, which jumps to the BIOS entrypoint (0xBFC00000), resets some COP0 registers and the general purpose registers, and resets some IO. Does not clear vram.
- Hard reset (Shift-F8): Similar to a reboot of the PSX.

2.3 Configuration



- Emulation : Emulation settings
- GPU : Graphics Processing Unit settings
- SPU : Sound Processing Unit settings
- UI : Change user interface settings (such as font size, language or UI theme)
- Controls : Edit KB/Pad controls
- Shader presets : Apply a shader preset
- Configure shaders : Show shader editor

2.4 Debug

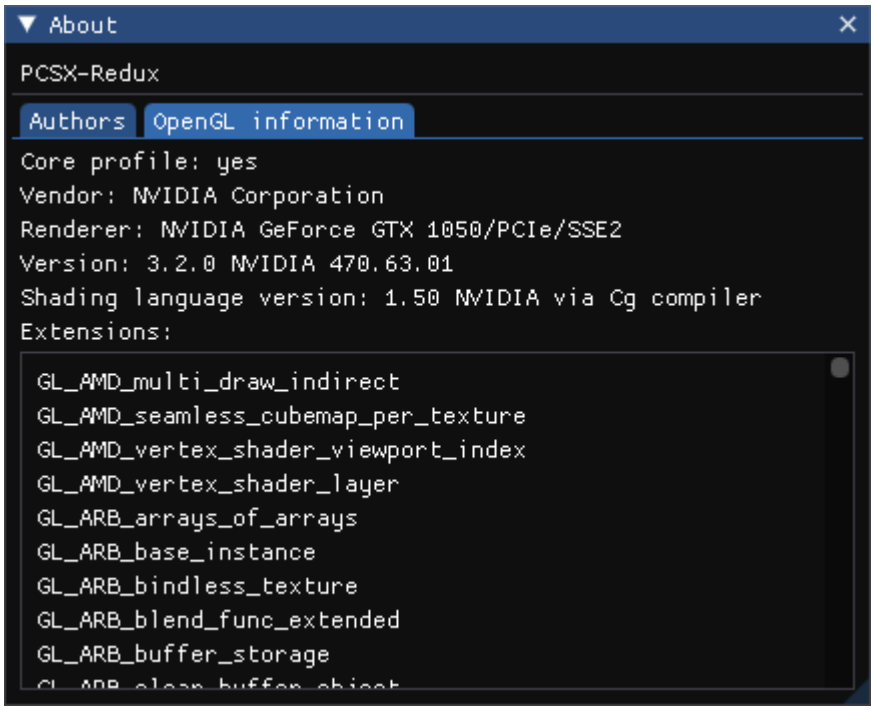


2.5 Help

- Show ImGui demo
- About

2.6 GPU information

The 'About' dialog available in the 'Help' menu has an 'OpenGL information' tab that displays information on the GPU currently used by the program, such as the supported OpenGL extensions.


About
PCSX-Redux
Authors
OpenGL information
Core profile: yes
Vendor: NVIDIA Corporation
Renderer: NVIDIA GeForce GTX 1050/PCIe/SSE2
Version: 3.2.0 NVIDIA 470.63.01
Shading language version: 1.50 NVIDIA via Cg compiler
Extensions:
GL_AMD_multi_draw_indirect
GL_AMD_seamless_cubemap_per_texture
GL_AMD_vertex_shader_viewport_index
GL_AMD_vertex_shader_layer
GL_ARB_arrays_of_arrays
GL_ARB_base_instance
GL_ARB_bindless_texture
GL_ARB_blend_func_extended
GL_ARB_buffer_storage

3. Compiling PCSX-Redux

3.1 Getting the sources

The only location for the source is on github. Clone recursively, as the project uses submodules:

.

3.2 Windows

Install Visual Studio 2019 Community Edition.
Open the file , select , right click, , and hit to build.
The project follows the open-and-build paradigm with no extra step, so no specific dependency ought to be needed, as NuGet will take care of downloading them automatically for you on the first build.

Note: If you get an error saying , you might need to delete the .suo file in vsproject/vs, restart Visual Studio and retry.

Openbios

Using Visual Studio Code, one can use the task "make_openbios" to compile: CTRL-P then to compile.

3.3 Linux

3.3.1 Compiling with Docker

Run . You need docker for this to work.

You will also need a few libraries on your system for this to work. Check the Dockerfile for a list of library packages to install.

3.3.2 Compiling with make

- Debian derivatives (for full emulator compilation):

- Arch derivatives :

You can then just enter the 'pcsx-redux' directory and compile without using docker with .

If you have a different mips compiler, you'll need to override some variables, such as .

Openbios

Building OpenBIOS on Linux can be done with docker : , or using , with the package installed ; .

3.3.3 MacOS

You need MacOS Catalina with the latest XCode to build, as well as a few homebrew packages.
Run the brew installation script to get all the necessary dependencies.

Run to build.

Compiling OpenBIOS will require a mips compiler, that you can generate using the following commands:

Openbios

Then, you can compile OpenBIOS using .

3.4 Compiling PSX code

If you're only interested in compiling psx code, you can clone the PCSX-Redux repo;

then install a mips toolchain and get the converted PsyQ libraries in the folder as per these instructions.

You can also find the pre-compiled converted Psyq libraries online.

3.4.1 Getting the toolchain on Windows

Download the MIPS toolchain here : https://static.grumpycoder.net/pixel/mips/g++-mipsel-none-elf-10.3.0.zip
and add the folder to your $PATH.
You can test it's working by launching a command prompt and typing . If you get a message like , then it's working !

3.4.2 Getting the toolchain on GNU/Linux

Debian derivative; Ubuntu, Mint...

Arch derivative; Manjaro...

The mipsel environment can be installed from AUR : cross-mipsel-linux-gnu-binutils and cross-mipsel-linux-gnu-gcc using your AURhelper of choice:

4. Command Line Flags

You can launch with the following command line parameters:

The parsing code doesn't care about the number of dashes in the parameter's flag, so '-' can be used as well as '--', or any number of dashes.

Flag	Meaning
	Dump the protobuf schemas for PCSX-Redux on stdout and exit immediately.
	Begin execution immediately on startup.
	Redirect log output to stdout.
	Redirect Lua's console output to stdout.
	Specify a file to log output to.
	Specify a BIOS file.
	Interpret internal API's command as a request to exit the emulator instead of pausing, and close the emulator. Implies , , and will also disable first chance exceptions. Use only when doing unit testing.
	Load a PSX exe.
	Load a PSX exe.
	Load a PSX disk image (iso, bin/cue).
	Load a PSX disk image (iso, bin/cue).
	Specify a memory card file to use as memory card slot 1.
	Specify a memory card file to use as memory card slot 2.
	Enable the pcdrv device interface. (Access PC filesystem through SIO).
	Specify base directory for pcdrv.
	Resets configuration to defaults.
	Resets the UI to its defaults.
	Enables kiosk mode, disabling UI interaction. Will change the saved setting.
	Disables kiosk mode, allowing the user to interact with the UI. Will change the saved setting.
	Use the interpreter CPU core.
	Use the dynamic recompiler CPU core.
	Activates the debugger. Will change the saved setting.
	Deactivates the debugger. Will change the saved setting.
	Skips the BIOS logo and boot animation. Will change the saved setting.
	Shows the BIOS logo and boot animation. Will change the saved setting.
	Activates the gdb server. Will change the saved setting.
	Deactivates the gdb server. Will change the saved setting.
	Sets the TCP port the gdb server is listening on. Will change the saved setting.
	Activates the CPU trace logging. Will change the saved setting.
	Deactivates the CPU trace logging. Will change the saved setting.
	Fully disables logs to be sent to the GUI.
	Specifies a .zip file to load for the function.

Flag	Meaning
	Specifies a Lua file to load through the function.
	Specifies a Lua string to execute.
	Enables Lua code coverage report. Requires the Lua module to be installed.
	Enables portable mode. Settings and saves will be stored in the same directory as the executable, or in the directory specified by the optional argument to this flag.

5. Debugging

5.1 Debugging with PCSX-Redux

PCSX-Redux has strong debugging capabilities. It has a built-in GDB server, which allows you to connect to it with a GDB client, such as gdb itself when targeting MIPS, a vscode connector, IDA Pro, or Ghidra, and debug the MIPS CPU. See debugging with Ghidra for more information on debugging with Ghidra.

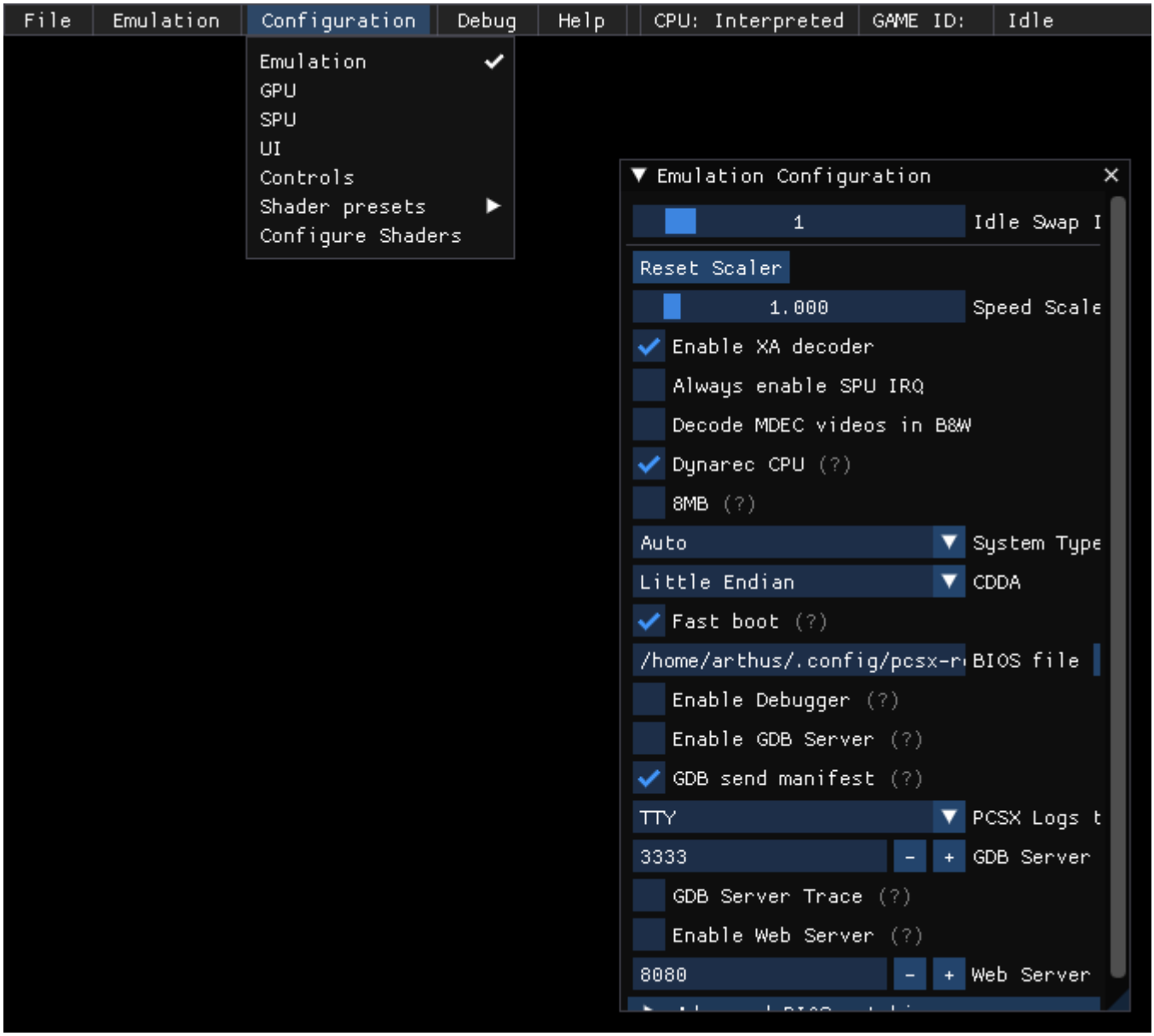
There are also built-in debugging tools, available in the Debug menu. Most of the CPU debugging features will require switching the Dynarec off from the Emulation configuration menu, as the Dynarec is not compatible with the debugging features. Additionally, the debugger needs to be enabled, also in the Emulation configuration menu.

The GPU debugging tools can work with the Dynarec enabled, and thus will be much faster than when the interpreter is used.

5.2 GDB server

The GDB server allows you to set breakpoints and control your PSX program's execution from your gdb compatible IDE.

5.2.1 Enabling the GDB server



In PCSX-Redux: .

Make sure the debugger is also enabled.

5.2.2 GDB setup

You need `` on your system :

Windows

Download a pre-compiled version from here : https://static.grumpycoder.net/pixel/gdb-multiarch-windows/

GNU/Linux

DEBIAN BASED

Install via your package manager :

```
```

ARCH BASED

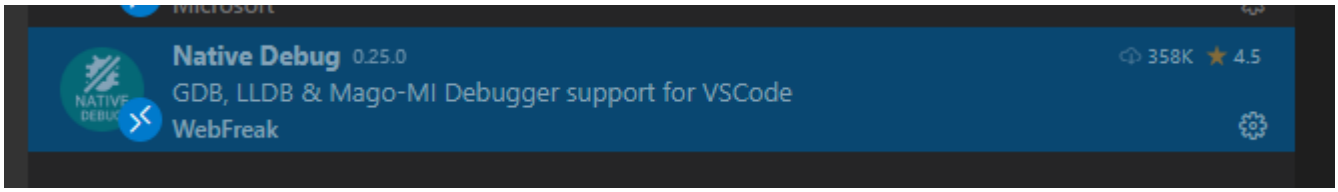
On Arch based distributions, multiarch is now enabled by default in regular builds and you don't need to install a specific version anymore.
You can install the 'gdb' package with `` :

```
```

5.2.3 IDE setup

MS VScode

- Install the `` extension : https://marketplace.visualstudio.com/items?itemName=webfreak.debug



- Adapt your file to your environment :
  A sample file is available here.
  This should go in .

You need to adapt the values of , and according to your system :

EXECUTABLE

This is the path to your executable :

https://github.com/grumpycoders/pcsx-redux/blob/a3bebd490388130e924124cdfeff3bc46b6149d9/.vscode/launch.json#L153

GDBPATH

This the path to the executable:

https://github.com/grumpycoders/pcsx-redux/blob/a3bebd490388130e924124cdfeff3bc46b6149d9/.vscode/launch.json#L154-L157

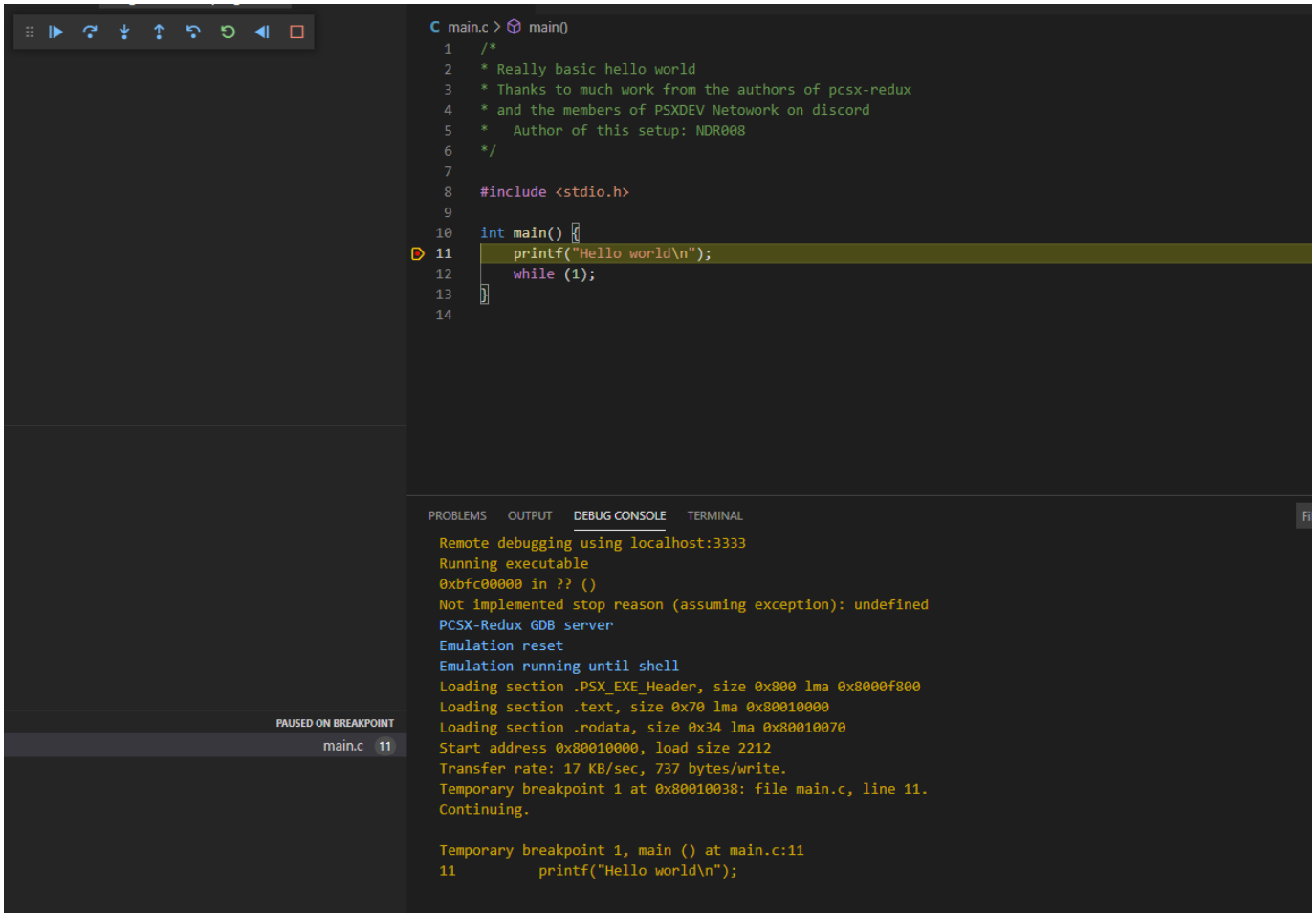
AUTORUN

Make sure that corresponds to the value.

https://github.com/grumpycoders/pcsx-redux/blob/a3bebd490388130e924124cdfeff3bc46b6149d9/.vscode/launch.json#L159-L165

By default, using should work, but if encountering trouble, try using your computer's local IP (e.g; 192.168.x.x, 10.0.x.x, etc.)

https://github.com/grumpycoders/pcsx-redux/blob/a3bebd490388130e924124cdfeff3bc46b6149d9/.vscode/launch.json#L150

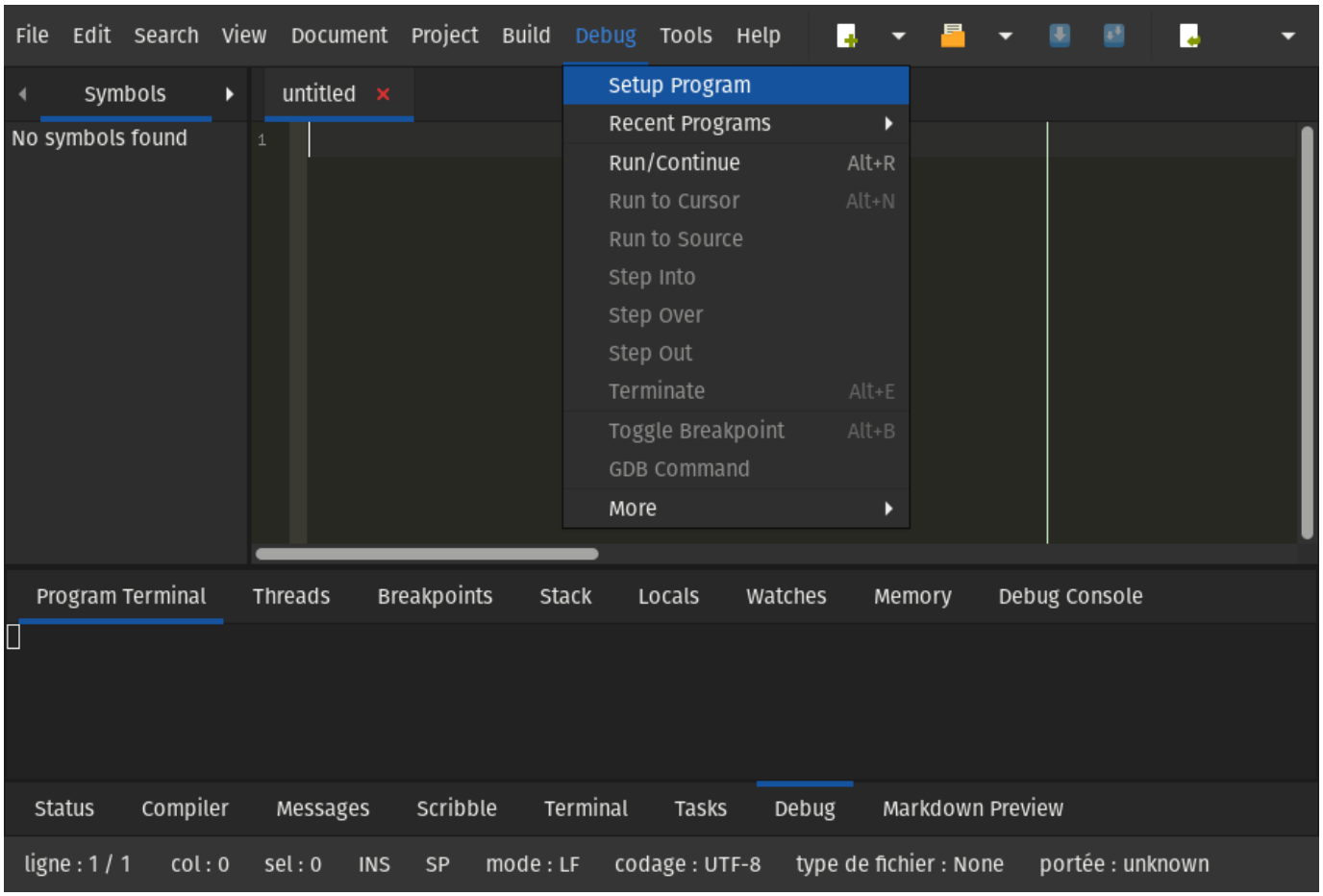


Geany

Make sure you installed the official plugins and enable the .

To enable the plugin, open Geany, go to and enable .

You can find the debugging facilities in the menu ;



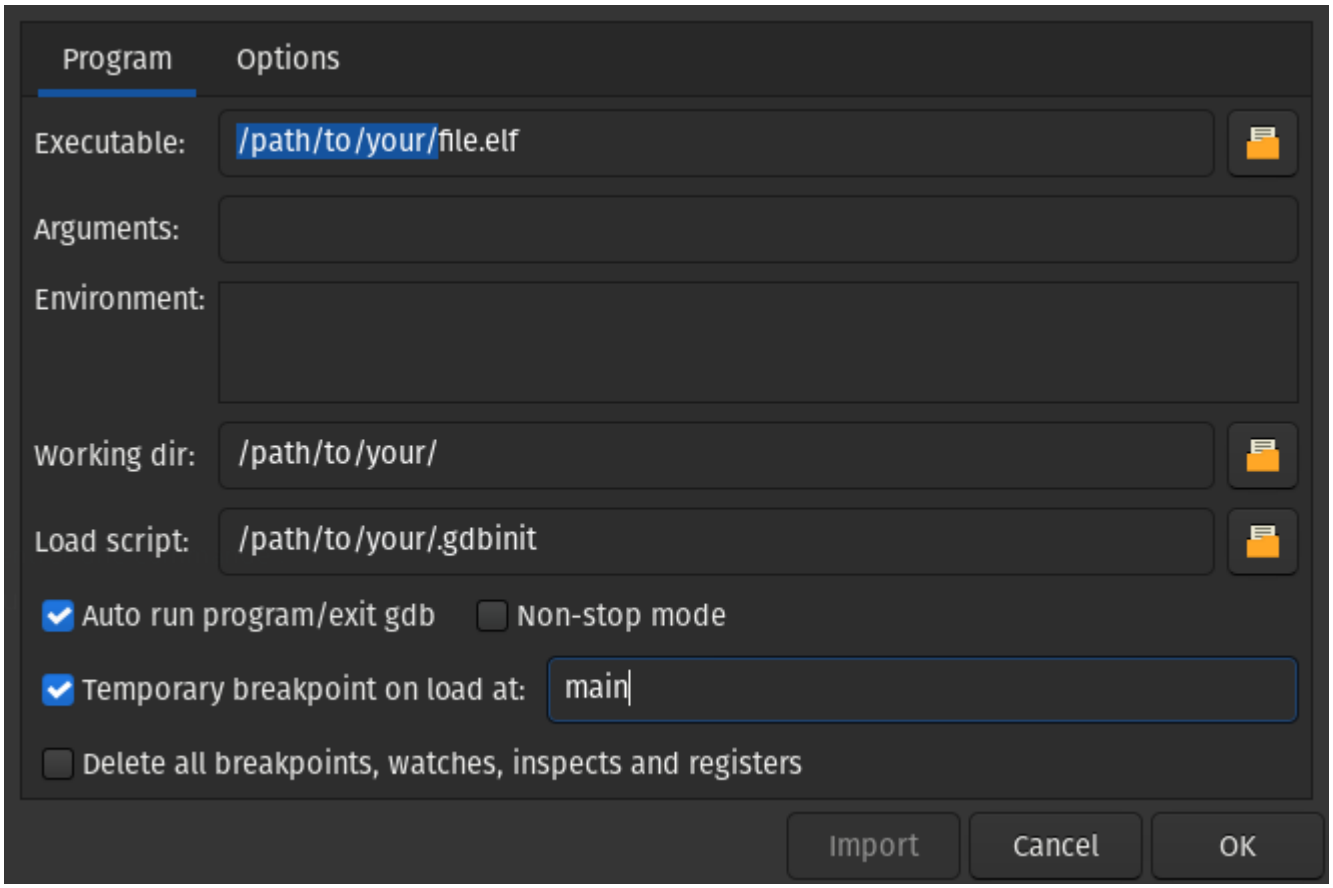
You can find the plugin's documentation here : https://plugins.geany.org/scope.html

.GDBINIT

Create a file at the root of your project with the following content, adapting the path to your file and the gdb server's ip.

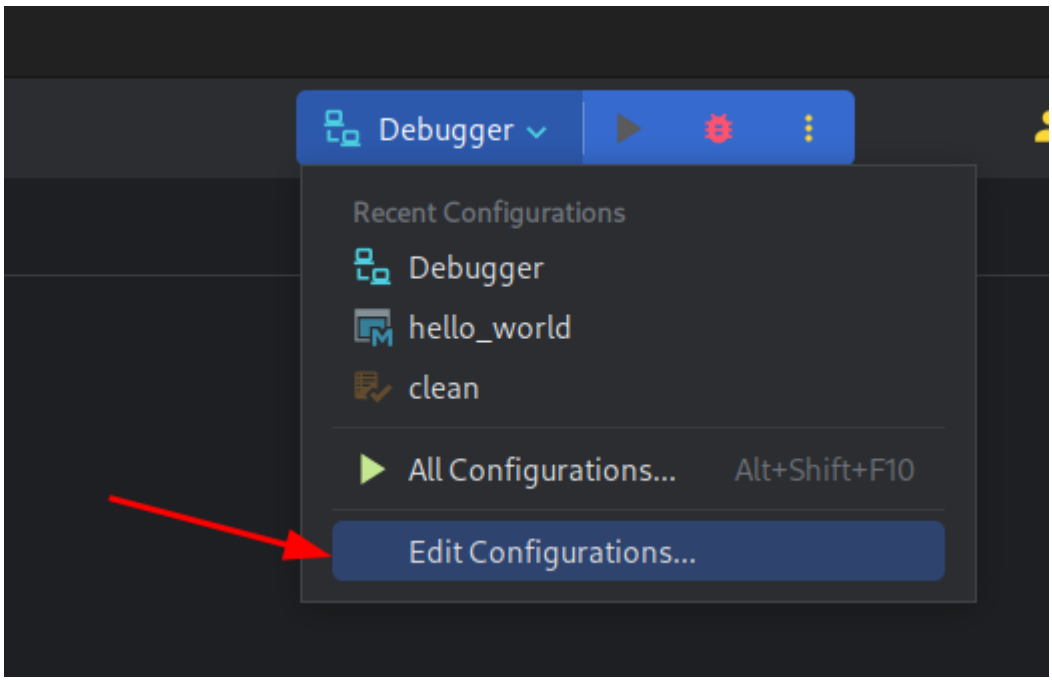
PLUGIN CONFIGURATION

In Geany : :

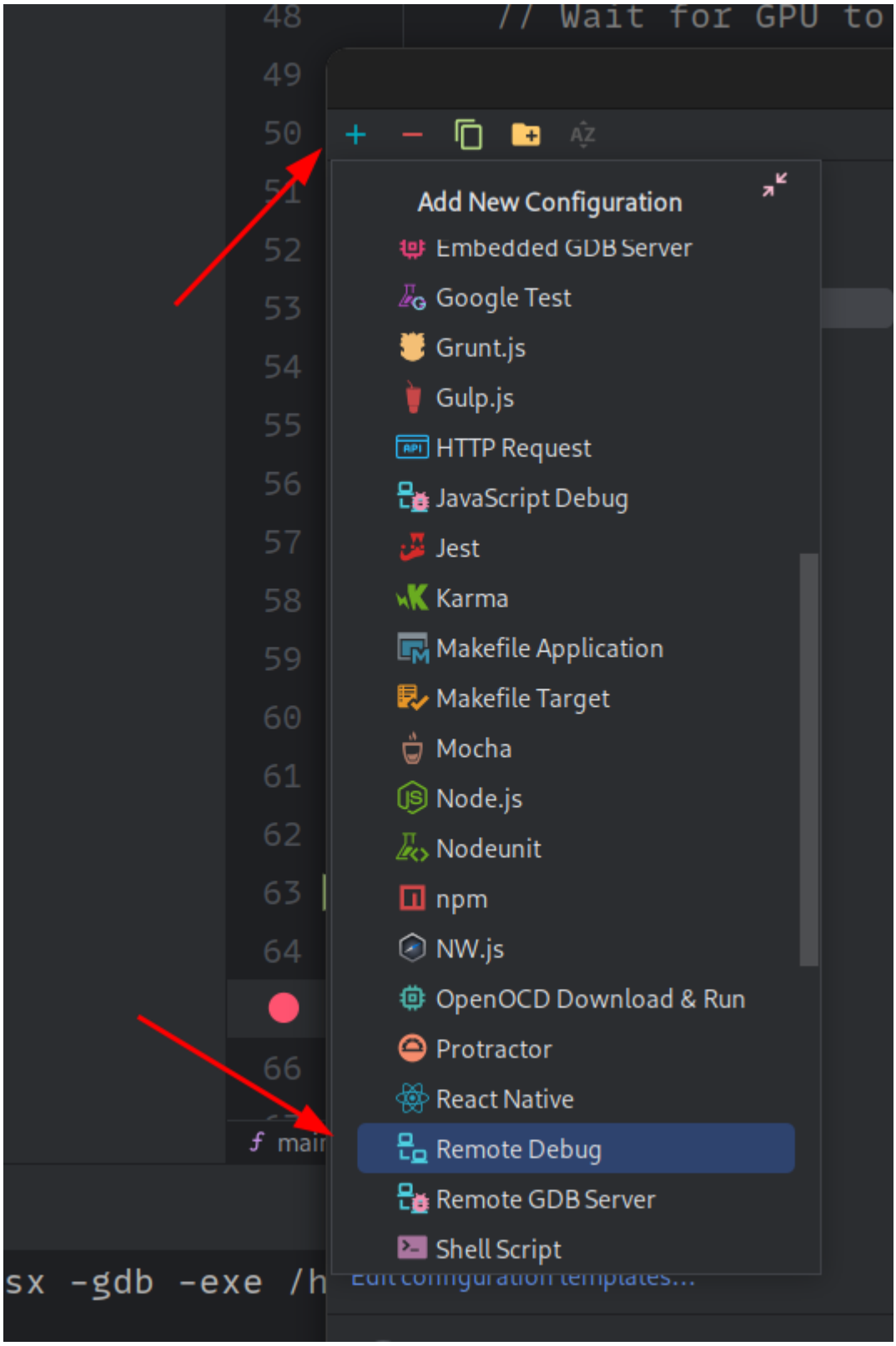


CLion

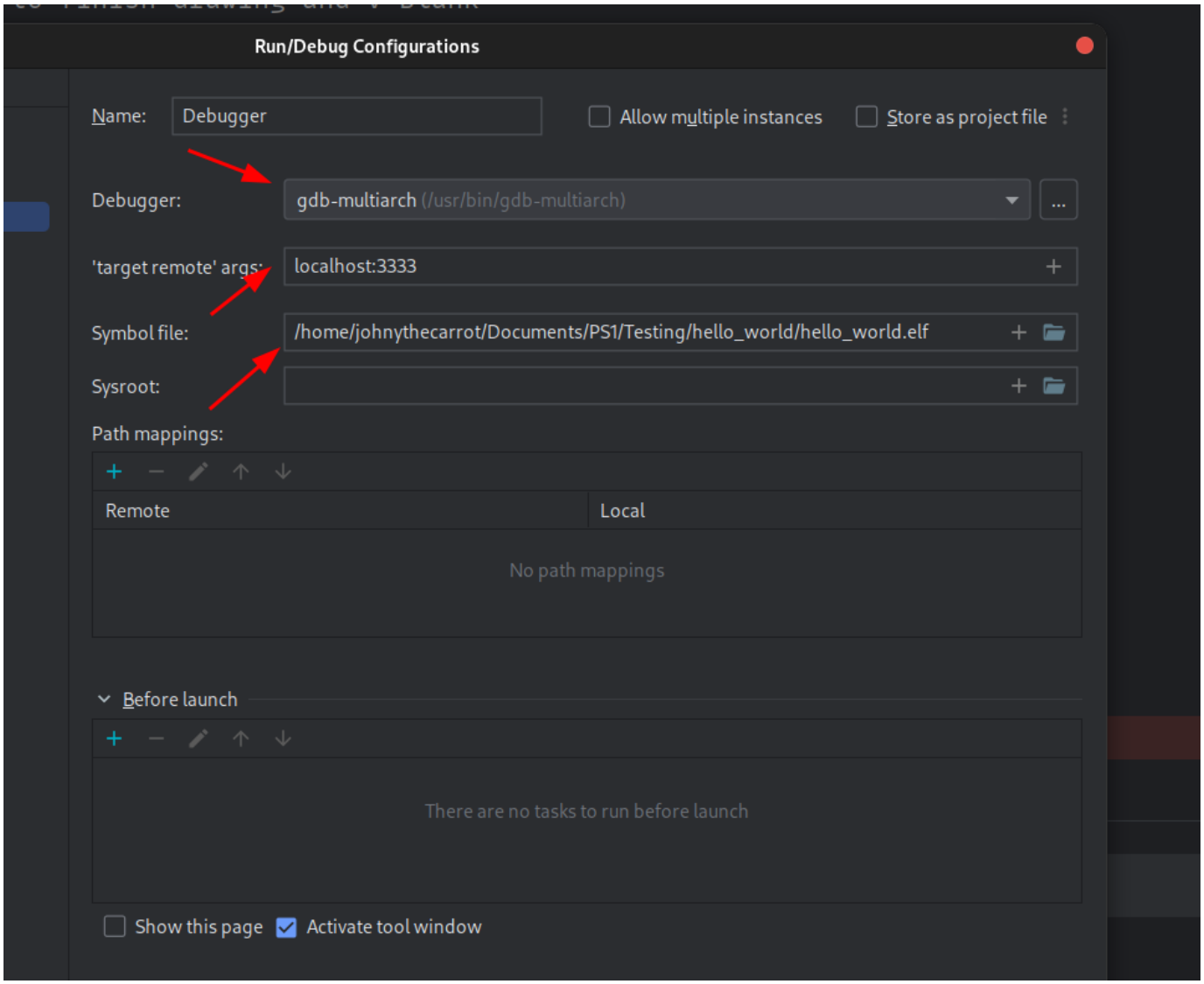
Open the Run/Debug Configurations menu, which you can find here:



Then, add a new Remote Debug configuration:



Finally, set your new configuration up:



.GDBINIT

Create a file at the root of your project with the following content, adapting the path to your file.

5.2.4 Beginning Debugging

Launch , then run the debugger from your IDE. It should load the file, and execute until the next breakpoint.

Starting debugging in Geany

Your browser does not support the video tag.

Source : https://archive.org/details/pcsx_redux_geany_gdb

5.2.5 Additional tools

https://github.com/cyrus-and/gdb-dashboard/

5.3 Connecting Ghidra to PCSX-Redux

Since version 10.3, Ghidra now supports debugging MIPS targets. This allows for a much more powerful reverse engineering experience than what was previously possible with the GDB server. This document will explain how to set up Ghidra to debug PCSX-Redux, as it is not entirely straightforward.

5.3.1 Prerequisites

- A gdb "multiarch" binary is required. For Windows, you can get it from here. For Linux, you can get it from your distribution's package manager; on Ubuntu and Debian, this is the package . And for MacOS, you can use the brew package manager to install it; this is the package .
- Ghidra 10.3 or newer. You can get it from here.
- PCSX-Redux either configured to disable Dynarec, enable the debugger, and enable the gdb server, or started using the following command-line arguments: .
- The following file downloaded somewhere on your computer, naming it .

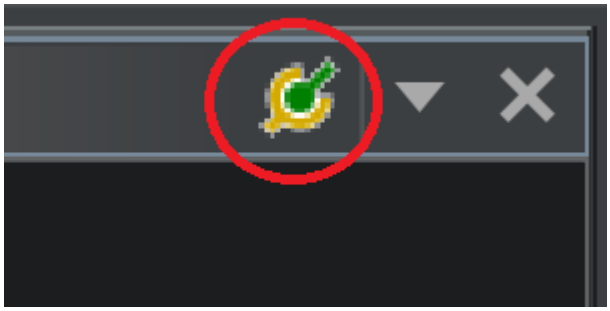
5.3.2 Setting up Ghidra

Before starting Ghidra, until version 10.3.3, the MIPS CPU isn't terribly well defined. One needs to go to the installation files of Ghidra, and edit the file . In this file, find the lines , and change them to . This will allow Ghidra to properly recognize the MIPS CPU used by the PlayStation 1. This step is no longer necessary starting with Ghidra 10.3.3.

5.3.3 Setting up Ghidra's debugger

When in the main view of Ghidra, right click on the project you want to debug, and in the context menu, select . This will open the debugger tool instead of the default disassembler tool.

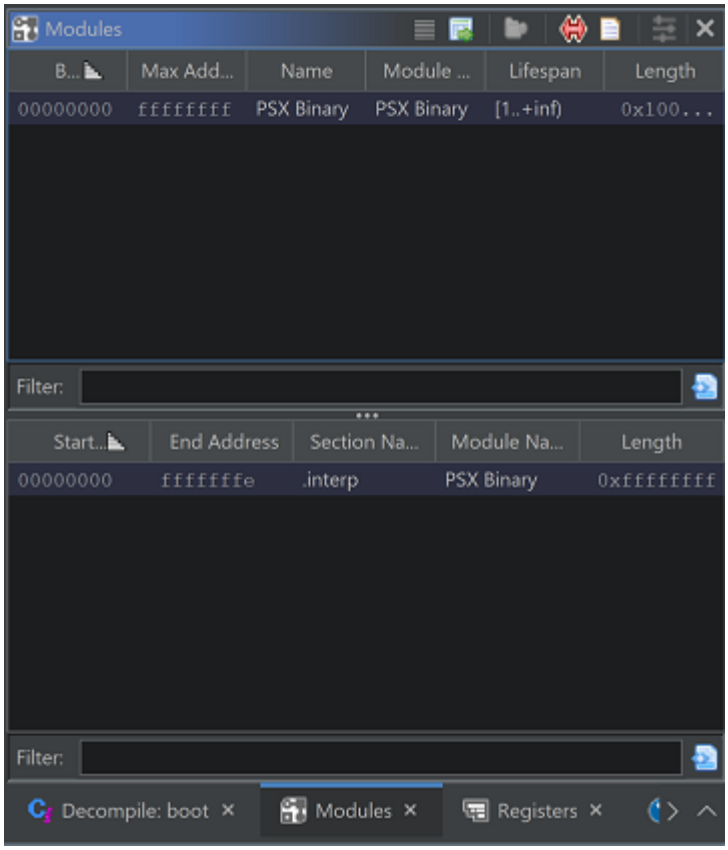
First, identify the Debugger Targets window, and click its top right button:



This will open the debugger connector window. In the drop down, select , and as the launch command, enter the path to the gdb multiarch binary, followed by . For example, on Windows, this could be . Click .

A new window should open on the right, with the prompt allowing you to type commands. First, you need to source the file from before. To do this, type . For example, on Windows, this could be . Then, you need to connect to the PCSX-Redux gdb server. To do this, type . Finally, locate the tab in

the right window, next to the tab, which should look like this:



Select the top line, right click on it, and in the context menu, select . In the new window that appears, simply click .

At this point, Ghidra should be fully connected to PCSX-Redux, and should be able to place breakpoint, resume or pause execution, inspect variables, etc. Please be aware that, as of Ghidra 10.3, many features of the debugger are still work in progress, and won't necessarily be stable.

5.4 Misc Features

5.4.1 Mapping breakpoints

PCSX-Redux has a feature that allows mapping the memory of the console while the software is running, and to set breakpoints on the mapped memory. This can for instance help in finding codepath when performing certain activities when running code.

First, map the kind of action you want to discover, such as executing code, reading memory, or writing memory. Then, run the code for some time without performing the specific action you want to discover. Finally, activate the map breakpoint mode, and then perform the action you want to discover. The breakpoint should be triggered when the action is performed.

For example, say that in a game, you want to know what code is executed when you press the "X" button. First, check the checkbox. Then, run the game for a while without pressing the "X" button. This will map enough of the memory that's being run in a normal way. Finally, activate the checkbox, and press the "X" button. If the game takes a new codepath that hasn't been executed yet, the breakpoint should be triggered.

Breakpoints are always checked before mapping the memory, so it's safe to keep both checkboxes on at the same time.

Click the button to zero out all of the maps, when starting anew.

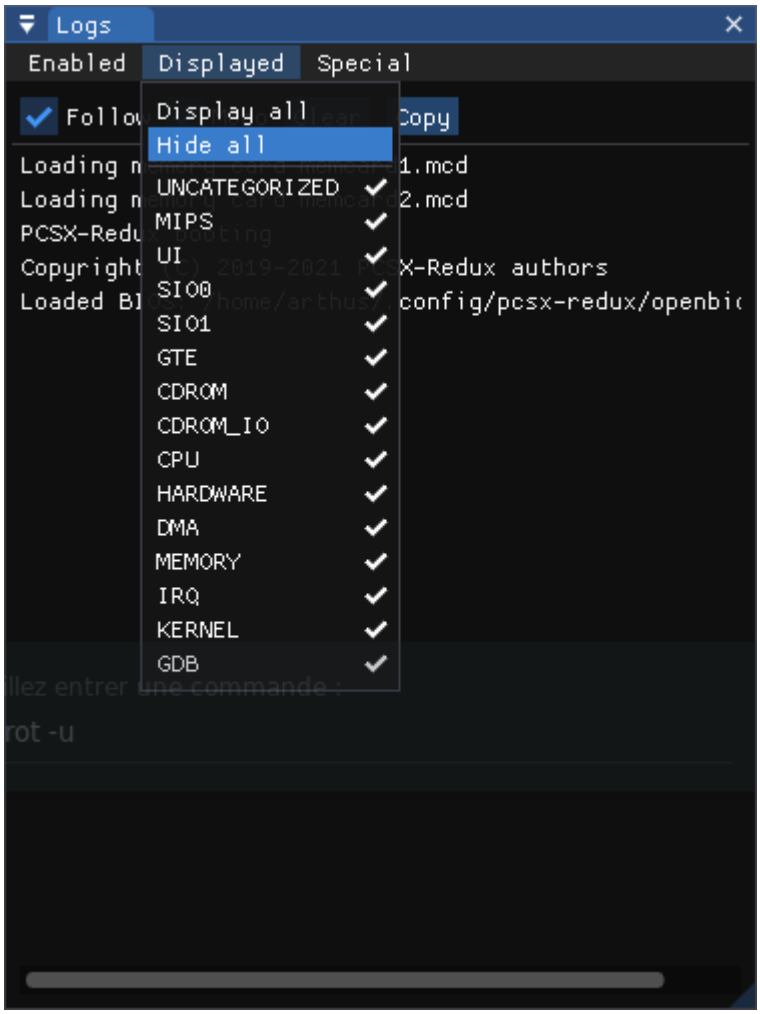
5.4.2 CPU trace dump

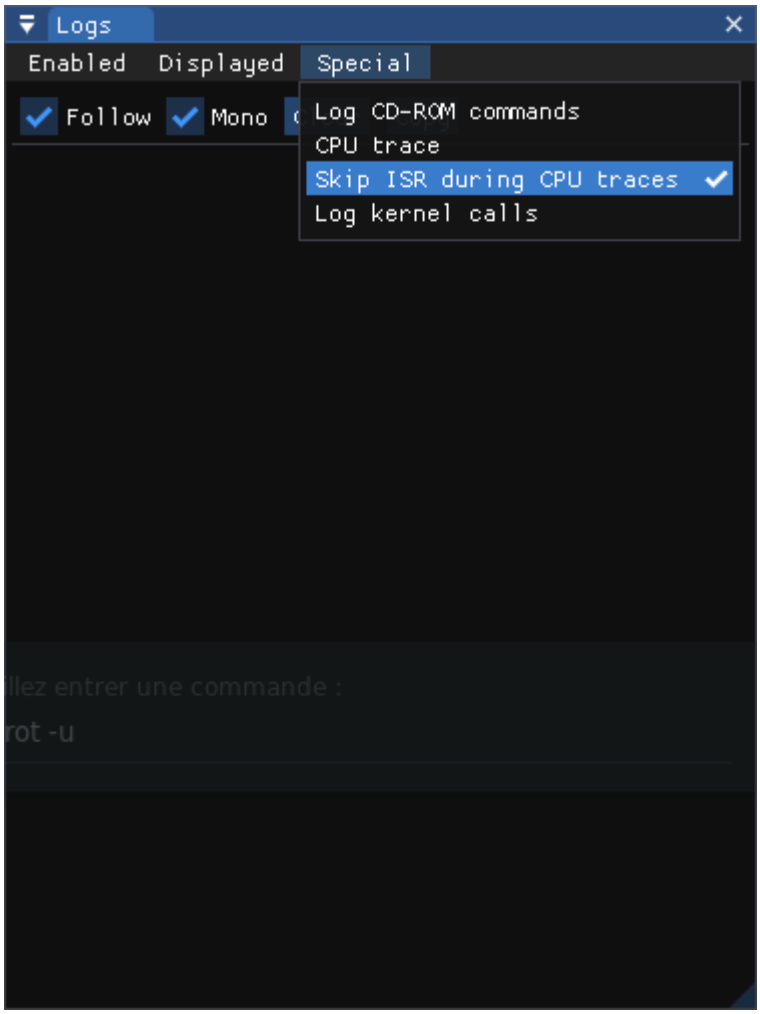
Setup

In PCSX-Redux, make sure is enabled.

In the 'Logs' window, hide all logs :

To avoid unnecessary noise, you can also skip ISR during CPU traces :


Logs
Enabled
Displayed
Special
Display all
Hide all
UNCATEGORIZED
MIPS
UI
SIO0
SIO1
GTE
CDROM
CDROM_IO
CPU
HARDWARE
DMA
MEMORY
IRQ
KERNEL
GDB
Copy



Begin dump

To dump the CPU traces, launch pcsx-redux with the following command :

```
```

You can use additional flags to launch an executable/disk image in one go, e.g :

```
```

Source

https://discord.com/channels/642647820683444236/663664210525290507/882608398993063997

5.5 VRAM viewer

5.5.1 Navigating

Holding the middle button, or both the left and right buttons, allows you to pan the view around. Using the wheel allows you to zoom in and out, at the location of the mouse cursor.

5.5.2 Lensing

Holding the CTRL key of your keyboard will bring up a lens, which will show you a locally zoomed version of the VRAM at the location of the mouse cursor. The lens can be resized by using the wheel while holding the CTRL key. Holding the CTRL and Shift buttons while using the wheel will change the size of the lens. The lens can be closed by releasing the CTRL key.

5.5.3 The various viewers

There are different viewers available from the main menu, which can be used to visualize the VRAM in different ways. The main viewer will let you see the VRAM using various CLUTs. The CLUT viewer will let you select a CLUT to use for the main VRAM viewer. In order to do this, first select the 8-bits or 4-bits view in the main viewer. Then, in the CLUT viewer, select . At this point, hovering the CLUT viewer will automatically change the main viewer to use the hovered CLUT. Once the proper view is found, simply click on the first pixel of the CLUT viewer to select the CLUT more permanently.

The GPU logger will also select CLUTs and change the main viewer's mode automatically, depending on the GPU commands being inspected.

5.6 GPU Logger

The GPU logger is a tool that allows you to see the GPU commands being executed by the emulator, and the resulting VRAM changes. It can be used to debug the GPU, and to understand how the executed software is rendering the scene. The logger will have a full frame worth of primitives, and will automatically clear the log when a new frame is started. Note that the notion of a frame may span over multiple vsyncs, if the PlayStation software isn't running at full FPS.

Note that it can be fairly resource intensive, and may significantly slow down the emulation, depending on the context.

The top of the GPU Logger window will have the following checkboxes:

- GPU Logging - Enable or disable the GPU logging.
- Breakpoint on vsync - Pause the emulation when a vsync occurs, allowing to inspect the current frame.
- Replay frame - Enables the replay of the current frame. See below for details.
- Show origins - Show the data path of the primitives. This will show the origin of the data, and the path it took to reach the GPU. For example, a sequence of primitives may be sent to the GPU via chained DMA.

5.6.1 Understanding the logs

The top of the logger can be expanded to display rough frame statistics. These values aren't necessarily too accurate, and are only meant to give a rough idea of the frame complexity.

Each row of the logger displays one command sent to the GPU. The first button and checkbox will be used for the replay system. The next three buttons and checkboxes will be used for the highlighting system. The next column will display the command name, and opening the tree node will expand the command parameters.

The expanded node may have buttons which will affect the main VRAM viewer, either by selecting CLUTs, or zooming in on the corresponding region. The VRAM viewer will also be updated when the replay system is used.

5.6.2 Highlighting Primitives

The GPU logger can highlight primitives in the VRAM viewer. One or more primitives may be selected, and the corresponding VRAM regions will be outlined. The highlighting will be cleared when a new frame is started. The default outlined colors will be red for written pixels, and green for read pixels. The colors can be changed in the main VRAM viewer settings.

Checking the checkbox will temporarily outline a primitive when hovering it in the logger. This can be useful to quickly identify the corresponding primitive in the VRAM viewer by flicking the mouse over the logger.

Checking the second checkbox in a logger node will permanently highlight the corresponding primitive in the VRAM viewer. The and buttons will select the beginning and the end of a span of primitives, and highlight them in the VRAM viewer.

5.6.3 Replay System

Once a frame has been logged properly, and the emulator is paused, the replay system can be used to replay the frame. The replay system will constantly replay the frame as long as it is activated, and it will update the VRAM viewer accordingly. By default, all nodes in the logger will be selected for replaying. Unselecting the first checkbox in a node will prevent it from being replayed, and the VRAM viewer will show what happens when this primitive isn't executed, and potentially see what is underneath it. Clicking the button of a node will select all nodes for replaying until this node, allowing to easily see the frame being built up to this point.

6. Mips API

6.1 Description

PCSX-Redux has a special API that mips binaries can use :

Source : https://github.com/grumpycoders/pcsx-redux/blob/main/src/mips/common/hardware/pcsxhw.h#L31-L36

The API needs DEV8/EXP2 (1f802000 to 1f80207f), which holds the hardware register for the bios POST status, to be expanded to 1f8020ff.
Thus the need to use a custom if you plan on running your code on real hardware.
The default file provided with the Nugget+PsyQ development environment does that:

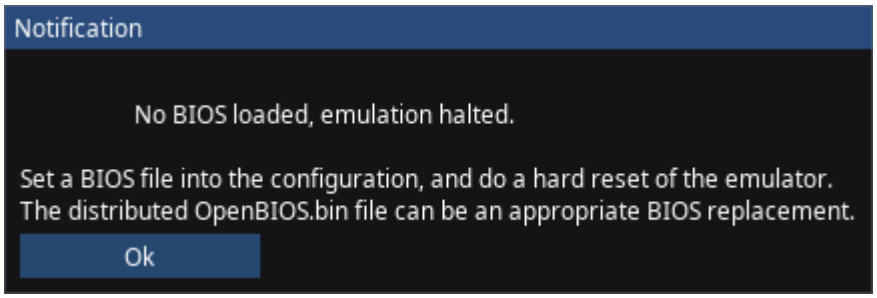
Source : https://github.com/grumpycoders/pcsx-redux/blob/main/src/mips/common/crt0/crt0.s#L36-L46

6.2 Functions

The following functions are available :

Function	Usage
	Print ASCII character with code to console/stdout.
	Break execution (Pause emulation).
	Executes Lua function at . The value can be between 1 and 255. If no Lua function exists within a slot, then this behaves the same as .
	Exit emulator and forward as exit code.
	Create a UI dialog displaying
	Enable or disable kernel checking.
	Returns truthy if kernel checking is enabled.
	Returns 1 if code is running in PCSX-Redux

Example of a UI dialog created with :



The kernel checking feature is used to try and catch unwanted accesses to the kernel, which are usually a sign of a bug in the code, such as a buffer overflow or a null pointer dereference. If the kernel checking feature is enabled, the emulator will break execution

and display a message in the console if it detects an unwanted access to the kernel. The following actions are considered unwanted accesses to the kernel:

- Reading or writing to a kernel address from a user-mode address and while not in a kernel-mode context such as while in the ISR. The ISR sets up a stack frame within the kernel space, so callbacks from the kernel and into the user space will be using kernel space as the stack. This means that a null pointer dereference in a callback from the kernel during an interrupt or exception will not be caught by the kernel checking feature.
- An indirect jump to a kernel address from a user-mode address and that isn't 0xa0, 0xb0, or 0xc0, and that isn't a instruction. Direct jumps and branches to kernel addresses should be compiler-level problems, so they are not checked for. The exception to the rule is because callbacks from the kernel will use to return to the kernel. Optimizations which bypass the instruction by using a different register to return to the kernel during a callback will cause false positives.

The feature is disabled by default as many games and software will access the kernel in various ways, and it can be enabled by calling . The feature can be disabled by calling . Since many startup sequences will access the kernel to patch it or clean it, it is recommended to enable the feature after the startup sequence has completed. Some libraries may also access the kernel during their normal operations. The user can simply disable the checker temporarily by toggling it before and after calling such APIs. The kernel space is considered to be all the memory addresses between 0x80000000 and 0x8000ffff. The BIOS is considered to be part of the kernel space in terms of code, so any access to the RAM Kernel space from the BIOS memory space will not trigger any of the kernel checks. The kernel checking feature is only available in the interpreter with the debugger enabled, and it is not available in the dynarec. Trying to enable the feature while using the dynarec, or while the debugger is disabled, will not have any effect.

7. Web server

A web server can be activated. This allows the use of a REST api to access various features. The server only handles up to HTTP/1.1, without SSL support.

7.1 Activation

You can activate the web server by going to

7.2 REST API

By default, the server listens for incoming connection on . The port can be changed in the same settings above.

These GET methods are available:

URL	Function
/api/v1/gpu/vram/raw	Dump VRAM
/api/v1/cpu/ram/raw	Dump RAM
/api/v1/execution-flow	Emulation Status
/api/v1/cd/files?filename=	Dump a file from the loaded disc image

The following POST methods are available:

The above needs to also send a form with binary contents. This will partially update the VRAM with the corresponding pixels. The updated rectangle has to be within the 1024x512 16bpp VRAM. The pixels need to be in 16bpp format, meaning the server is expecting exactly bytes in the form data. The server will properly parse requests with , but raw bytes in the request body without this header is also acceptable. Any invalid query will result in a 400 error.

The above needs to also send a form with binary contents, which will update the RAM at the specified offset. Offset is expected to be a number from [0, 0x1FFFFF] in case of running redux with 2MB RAM, or [0, 0x7FFFFF] in case the 8MB memory expansion is enabled. The value of size + offset must not exceed the total space in the RAM.

Value	Function
reset	Resets the symbols loaded in redux
upload	Uploads a file to redux

The above expects a file with symbols and addresses, which will be merged with the current symbols already loaded in redux. The map file should contain a pair of for each line. e.g would load the symbol in the address .

Value	Function
flush	Flushes the CPU cache

Value	Type	Function
pause	-	Pauses the emulator.
start	-	Starts/Resumes the emulator.
resume	-	Starts/Resumes the emulator.
reset	hard	Hard resets the emulator. Equivalent to a power cycle of the console.
reset	soft	Soft resets the emulator. Equivalent to pressing the reset button.

The above needs to also send a form with binary contents, which will patch the currently loaded iso file with the contents of the form. The server will look for the given filename in the iso file, and patch its contents. All changes are cumulative. If the file is not found, a 404 error will be returned. The file name is case sensitive, and must be a valid ISO9660 filename, which means it can only contain uppercase letters, numbers, and underscores, and ends with .

For example:

The above needs to also send a form with binary contents, which will patch the currently loaded iso file with the contents of the form. The iso sectors starting at the given value will be written to. The argument is optional, and can be of the following values:

Value	Function
GUESS	Tries to guess the sector's mode. This is the default.
RAW	Writes the full sectors with no decoration, 2352 bytes per sector.
M2_RAW	Writes 2336 bytes per sector, with the first 16 bytes being the subheader.
M2_FORM1	Writes 2048 bytes per sector. Will not update the subheader.
M2_FORM2	Writes 2324 bytes per sector. Will not update the subheader.

All changes are cumulative.

Value	Function
save	Saves the current state of the disc image patches to a PPF file.
clear	Clears the current list of patches.

8. Lua

8.1 Introduction

PCSX-Redux features a Lua API that is available through either a direct Lua console, or a Lua editor, both available through the Debug menu. The Lua VM runs on the main thread, the same one as the UI and the emulated MIPS CPU. As a result, care must be taken to not stall for too long, or the UI will become unresponsive. Using coroutines to handle long-running tasks is recommended, yielding periodically to let the UI perform some work too. The UI is probably going to run at 60FPS or so, which gives a ballpark of 15ms per frame.

8.1.1 Lua engine

The Lua engine that's being used is LuaJIT 2.1.0-beta3 compiled in Lua 5.2 compatibility mode. The Lua 5.1 user manual and LuaJIT user manual are recommended reads. In particular, the bindings heavily make use of LuaJIT's FFI capabilities, which allows for direct memory access within the emulator's process. This means there is little protection against dramatic crashes the LuaJIT FFI engine can cause into the emulator's process, and the user must pay extra attention while manipulating FFI objects. Despite that, the code tries as much as possible to sandbox what the Lua code does, and will prevent crashes on any recoverable exception, including OpenGL and ImGui exceptions.

8.1.2 Lua console

All of the messages coming from Lua should display into the Lua console directly. The input text there is a single line execution, so the user can type one-liner Lua statements and get an immediate result.

8.1.3 Lua editor

The editor allows for more complex, multi-line statements to be written, such as complete functions. The editor will by default auto save its contents on the disc under the filename , which can potentially be a problem if the last statement typed crashed the emulator, as it'll be reloaded on the next startup. It might become necessary to either edit the file externally, or simply delete it to recover from this state.

The auto-execution of the editor permits for rapid development loop, with immediate feedback of what's done.

For complex projects however, it is recommended to split your work into sub-modules, and use the function to load them in your main code. This implies working on your project using an external editor.

8.2 Loaded libraries

8.2.1 Basic Lua

The LuaJIT extensions are fully loaded, and can be used globally. The standard Lua libraries are loaded, and are usable. The function exists, but isn't recommended as the loading of external DLLs might be difficult to properly accomplish. Loading pure Lua files is fine. The table is loaded globally, there is no need to it, but it'll work nonetheless. As a side-effect of Luv, Lua-compat-5.3 is loaded.

8.2.2 Dear ImGui

A good portion of ImGui is bound to the Lua environment, and it's possible for the Lua code to emit arbitrary widgets through ImGui. It is advised to consult the user manual of ImGui in order to properly understand how to make use of it. The list of current bindings can be found within the source code. Some usage examples will be provided within the case studies. Additional features and interaction is documented in the rendering page.

8.2.3 OpenGL

OpenGL is bound directly to the Lua API through FFI bindings, loosely inspired and adapted from LuaJIT-OpenCL. Some usage examples can be seen in the CRT-Lottes shader configuration page.

8.2.4 NanoVG

The NanoVG library is mostly bound to the Lua API through FFI bindings, with some additional glue code. More explanation can be found in the rendering page.

8.2.5 Luv

For network access and interaction, PCSX-Redux uses libuv internally, and is exposed to the Lua API through Luv, tho its loop is tied to the main thread one, meaning it'll run only once per frame. There is another layer of network API available through the File API, which is more convenient and faster for simple tasks.

8.2.6 Zlib

The Zlib C-API is exposed through FFI bindings. There is another layer of Zlib API available through the File API, which is more convenient and faster for simple tasks.

8.2.7 FFI-Reflect

The FFI-Reflect library is loaded globally as the symbol. It's able to generate reflection objects for the LuaJIT FFI module.

8.2.8 PPrint

The PPrint library is loaded globally as the symbol. It's a more powerful function than the one provided by Lua, and can be used to print tables in a more readable way.

8.2.9 Lua-Protobuf

The Lua-Protobuf library is available, but not loaded by default. All of its documented API should be usable straight with no additional work. It has been slightly modified, but nothing that should be visible to the user. There is some limited glue between its API and PCSX's.

8.2.10 luafilesystem

The luafilesystem library is loaded globally as the symbol. It's a library that provides access to the filesystem.

8.2.11 LPeg

The LPeg library is available, but not loaded by default. It's a library that provides a pattern-matching library for Lua, which can be useful to create ad-hoc arbitrary parsers.

8.3 Redux basic API

8.3.1 Settings

All of the settings are exposed to Lua via the table. It contains pseudo-tables that are reflections of the internal objects, and can be used to read and write the settings. The exact list of settings can vary quickly over time, so making a full list here would be fruitless. It is possible however to traverse the settings using for example. The semantic of the settings is the same as from within the GUI, with the same caveats. For example, disabling the dynamic recompiler requires a reboot of the emulator.

8.3.2 ImGui interaction

PCSX-Redux will periodically try to call the Lua function to allow the Lua code to draw some widgets on screen. The function will be called exactly once per actual UI frame draw, which, when the emulator is running, will correspond to the emulated GPU's vsync. If the function throws an exception however, it will be disabled until recompiled with new code.

8.3.3 Events Engine interaction & Execution Contexts

LuaJIT C callbacks aren't called from a safe execution context that can allow for coroutine resuming, and luv's execution context doesn't have any error handling.

It is possible to defer executing code to the main loop of PCSX-Redux, which can (a) resume coroutines and (b) execute code in a safe context. The function

will execute the given function in the next main loop iteration. Here's some examples of how to use it:

Of course, this can also delay processing significantly, as the main loop is usually bound to the speed of the UI, which can mean up to 20ms of delay.

8.3.4 Constants

The table contains numerical constants used throughout the rest of the API. Keeping an up to date list here is too exhausting, and it's simpler to print them using .

8.3.5 Pads

You can access the pads API through where is the slot number and is the pad number, both indexed from 1, Lua-style. So accesses the first pad, and accesses the second pad.

Each Pad table has the following functions:

The button constants can be found in .

You can for instance press the button Down on the first pad using the following code:

8.3.6 Execution flow

The Lua code has the following API functions available to it in order to control the execution flow of the emulator:

-
-
-
-

It's also possible to manipulate savestates using the following functions:

-
-
-

Additionally, the following function returns a string containing the .proto file used to serialize the savestate:

-

Note that the actual savestates made from the UI are gzip-compressed, but the functions above don't compress or decompress the data, so if trying to reload a savestate made from the UI, it'll need to be decompressed first, possibly through the zReader File object.

Overall, this means the following is possible:

8.3.7 Messages

The globals and are available, and will display logs in the Lua Console. You can also use to display a line in the general Log window. All three functions should behave the way you'd expect from the normal function in mainstream Lua.

8.3.8 GUI

You can move the cursor within the assembly window and the first memory view using the following functions:

-
-

8.3.9 GPU

You can take a screenshot of the current view of the emulated display using the following:

-

This will return a struct that has the following fields:

The will contain the raw bytes of the screenshot data. It's meant to be written out using the method on a object. The and will be the width and height of the screenshot, in pixels. The will be either or , depending on the color depth of the screenshot. The size of the Slice will be multiplied by the number of bytes per pixel, depending on the .

8.3.10 Loading and executing code

While the basic Lua functions and exist, some alternative functions are available to load and execute code in a more flexible way.

- will load the given zip file, and will make it available to the function. It is equivalent to the command line flag. Note that if a file named is found in the zip file, it will be executed automatically.
- will load the given file, and execute it. It is equivalent to , but will also search for the file next to the currently loaded Lua file which is calling this function, and will also search for the file in all of the loaded zip files, either through the command line, or through the function.
- will load the given file, and return a function that can be called to execute the file. It is equivalent to , but has the same file search algorithm as .
- will open the given file as read only, and return a object. It is roughly equivalent to , but has the same file search algorithm as .

If given the following directory structure:

If contains the following code:

Then running the following code:

Will first load from the zip file , run it, which will in turn load from the zip file again, and execute it.

This allows distributing complex "mods" as zip files, which can be loaded and executed from the command line or the console.

8.3.11 Miscellaneous

- schedules the emulator to quit. It's not instantaneous, and will only quit after the current block of Lua code has finished executing, which will be before the next main loop iteration. The parameter is optional, and will be the exit code of the emulator. If not specified, it'll default to 0.
- returns an unsigned 64-bit number indicating how many CPU cycles have elapsed. This can be paired with the constant to determine how much emulated time has passed.
- will return an Adpcm encoder object. The object has the following methods:
- will reset the encoder, and set the mode to the given mode. The mode can be , , , , . The default mode is , which enables all the filters available in the SPU. The mode limits the encoder to the filters available in the XA ADPCM format. The mode uses the high-pass filter, and the mode uses the low-pass filter. The mode forces plain 4-bit Adpcm encoding.
- will encode the given ffi input buffer, and write the result to the given ffi output buffer. The input buffer should be a buffer of 16-bit signed integers, and the output buffer should be a buffer of 16-bit signed integers. The channels parameter is optional, and will default to 2. The input buffer should contain exactly 28 samples, and so does the output buffer. If the output buffer is not given, the function will return a new buffer with the result. LuaBuffers are also accepted as input and output buffers. The function will return three values: the output buffer, the filter index used, and the shifting used. The function is intended to be used as an intermediate computation step, and the output still needs to be processed into 4 bits or 8 bits samples.
- will encode the given ffi input buffer, and write the result to the given ffi output buffer. The input buffer

should be a buffer of 16-bit signed integers, and the output buffer should be a buffer which is at least 16 bytes large. The blockAttribute parameter is optional, and will default to . The input buffer should contain exactly 28 samples. If the output buffer is not given, the function will return a new buffer with the result. LuaBuffers are also accepted as input and output buffers. The function will return the encoded block, suitable for SPU usage. The parameter can be one of the following strings: , , , , .

- will write the opinionated end of sample looping block, as prescribed by the original Sony API. The output buffer should be a buffer which is at least 16 bytes large. If the output buffer is not given, the function will return a new buffer with the result. LuaBuffers are also accepted as output buffers. The function will return the encoded block, suitable for SPU usage.
- will encode the given ffi input buffer, and write the result to the given ffi output buffer. The input buffer should be a buffer of 16-bit signed integers, and the output buffer should be a buffer which is at least 128 bytes large. Note that a MODE2 FORM2 XA sector requires subheaders and 18 of these blocks. The xaMode parameter is optional, and will default to . The other valid value is . It will defines the encoding output between either 4-bit and 8-bit. The channels parameter is optional, and will default to 1. If the output buffer is not given, the function will return a new buffer with the result. LuaBuffers are also accepted as input and output buffers. The function will return the encoded block, suitable for XA usage. The amount of required input samples varies depending of the number of channels and the encoding mode:
- 4-bit mono: 224 samples aka 448 bytes
- 4-bit stereo: 112 samples aka 448 bytes
- 8-bit mono: 112 samples aka 224 bytes
- 8-bit stereo: 56 samples aka 224 bytes

Using the encoder to process an input audio file is as simple as:

8.4 Rendering

PCSX-Redux is entirely running as an OpenGL3 application. All of its aspects, including the UI elements, are rendered using OpenGL primitives. This means there is very little boundaries between the various rendered elements on the screen.

The rendering of the UI is done through ImGui, and a chunk of its API is bound is to Lua using bindings.

A good portion of the OpenGL3 API is also bound to Lua, as well as the nanovg library.

8.4.1 Emulated GPU rendering pipeline

The content of the Output region is rendered in two steps. The first step is called the "Offscreen rendering", and is done during the emulated GPU vsyncs. Its job is to flush the contents of the VRAM texture to an offscreen texture, which may be of a different resolution. The resolution of the offscreen texture should be pixel perfect with that of the Output region. By default, the associated shader with this operation should only do a simple copy and interpolation, but as the first stage of the rendering pipeline, this can be used for some first pass output effect such as the first pass of a crt shader.

The second step is called the "Output rendering", and is done every time the UI wants to refresh its display, which may or may not be at the same time as the emulated vsync. The resolution of the input will match exactly the resolution of the input texture, and the default shader should simply copy all the texels without any sort of interpolation, but as the second stage of the rendering pipeline, this can still be used for the second pass output effect.

The crt-lottes implementation leverages these two passes to do the full CRT-like output.

8.4.2 Shader editor

The shader editor is a simple text editor that allows to edit the shader code. It is not a full IDE, and it is not meant to be. Its point is to do quick iterations on the shader code, and to be able to see the result of the changes in real time.

The shader editor is split in 3 regions:

- The left tab is the vertex shader code. It is technically editable, but there shouldn't be much reason to edit it.
- The middle tab is the fragment shader code. This is the main shader code. It is editable, and the changes will be reflected in real time.
- The right tab is the Lua invoker code. This is the code that will be executed under multiple circumstances. It is editable, and the changes will be reflected in real time.

The Lua invoker code will be compiled and executed in a soft sandbox environment. The code can still access already created globals and mutate them, but any newly created global will be kept within the sandbox and won't be accessible from other Lua code. All these globals will be saved and restored with the normal emulator settings.

When the shaders are compiled, the Vertex and Fragment shader code will be compiled together, and if the resulting program is valid, the Lua invoker code will be compiled and executed. If the Lua code fails to compile or execute, the shader will be considered invalid and the error will be displayed in the shader editor.

This compilation order allows the Lua code to access the shader program uniforms, and to set them up as needed. The global will be available to the Lua code, and will contain the ID of the shader program.

The code is expected to export a few functions:

- , which will be called periodically within the ImGui context, allowing to draw UI elements. The global will be set to true when the user selects the "Configure Shaders" menu item. This allows to display a configuration UI to the user during this function call.
- , which will be called periodically within the ImGui context, when the emulator needs to draw the texture at the given size. The texture ID is the OpenGL texture ID, and the size is in pixels. The code is at best expected to do a simple call to to draw the texture. For the Emulated GPU Pipeline, this function will only be called on the Output shader, when being drawn to the Output region. As the function will be called during the ImGui context, it can capture certain ImGui state, such as the current ImGui cursor position, and use it to draw additional UI elements. Note that as with any normal ImGui function, this isn't the moment when the UI elements are actually drawn, but rather when the UI elements are queued to be drawn, meaning this isn't when the shader program will be executed, which is the point of the next function.
- will be called when the shader program is about to be executed, and needs to bind the attributes. The texture ID is the OpenGL texture ID, and the shader program ID is the OpenGL shader program ID. The location and sizes are in pixels, but are only used for the Emulated GPU Pipeline, when the Offscreen shader is being executed, as it needs to grab a portion of the VRAM texture to be rendered to the offscreen texture.

Additionally, it is possible to programmatically set the content of the editors using the following methods:

The method will set the default shader code, and the methods will set the shader code to the given string. The argument can be either an actual string, or a object.

8.4.3 ImGui

The ImGui API is bound to Lua, and can be used to draw UI elements. The ImGui API is documented on the ImGui source code. There is also an interactive manual available.

Not all functions are necessarily bound to Lua, and one can check the bindings code to see which functions are bound, and why some functions are not bound.

The main reason for not binding a function is that its arguments or return values are not trivial to bind. For example, the C++ function is not bound, as it takes a variadic number of arguments, which is not possible to bind in Lua easily. Instead, the C++ function is bound, which takes a single string argument.

The emulator will periodically try to call the global function with no arguments. If the function is not defined, nothing will happen. If the function fails to execute, it will be removed from the global environment, and the emulator will stop trying to call it until a new global is defined.

The function is expected to call the function to create a new ImGui window, as there is no default window created by the emulator for the Lua context. The function is also expected to call the function as normal with the ImGui API.

Some extra functions are bound to Lua beyond the API listed above:

- will create a new FFI object. The object is a simple struct with two fields, and . The function takes two optional arguments, the and values, and returns the new object.
- will return the current viewport ID. Viewports in ImGui are a way to split the ImGui context into multiple independent contexts, and the viewport ID is a unique identifier for each viewport. Basically, each viewport is a physical window from the operating system, and it can contain one or more ImGui windows.
- will return the viewport flags for the specified viewport. The viewport flags are of the type in the ImGui C++ API, and is a bitmask of flags, which are exposed as individual values in the Lua generated bindings.
- will set the viewport flags for the specified viewport. The proper usage of this function is to call to get the current flags, modify the flags as needed, and then call to set the new flags.
- will return the position of the specified viewport. The position is returned as an object.
- will return the size of the specified viewport. The size is returned as an object.
- will return the work position of the specified viewport. The work position is returned as an object.
- will return the work size of the specified viewport. The work size is returned as an object.
- will return the DPI scale of the specified viewport. The DPI scale is returned as a number. A value of 1.0 means that the DPI scale for this viewport is 100%.
- will create an input text widget. The is the label to display next to the input text, and the is the current text to display in the input text. The are optional, and are the same flags as the ones used by the C++ function. The function will return a boolean indicating if the text has changed or not, and the new text.

- will create an input text widget. The is the label to display next to the input text, and the is the hint to display in the input text when the text is empty. The is the current text to display in the input text. The are optional, and are the same flags as the ones used by the C++ function. The function will return a boolean indicating if the text has changed or not, and the new text.
- will call the C++ function, which will add the given text to current log buffer.
- will call the C++ function with the proportional font. It will need to be followed by a call to .
- will call the C++ function with the monospace font. It will need to be followed by a call to .

Safety

The ImGui API will frequently assert and crash the process if the API calls are imbalanced. For example, if the function is called without calling the function, the process will most likely crash.

This can be problematic when using the ImGui API from Lua, as the Lua code is not able to catch the crash, and the process will crash without any indication of what went wrong.

The main reason for imbalanced API calls can be attributed to the user code throwing an exception, which will cause the Lua code to unwind the stack, and the ImGui API will not be able to properly clean up its state.

For example, consider the following code:

The function will be called, but the function will not be called, as the function will unwind the stack, and the function will never be called.

In order to mitigate this, safe wrappers are provided for all of the ImGui Begin*/End* functions. The safe wrappers will catch any exception thrown by the user code, and will call the corresponding End* function if the Begin* function returned true. The error will be rethrown after the End* function is called. The wrapped lambda will only be called if the Begin* function returned true.

The example above can be rewritten as:

8.4.4 NanoVG

The NanoVG library is bound to Lua, and can be used to draw arbitrary vector graphics on top of the emulator. The NanoVG API is documented on the NanoVG source code. The API is very similar to the HTML5 Canvas API, meaning that one can use the MDN CanvasRenderingContext2D documentation and other related documentation to learn how to use it.

Using an HTML5 canvas toybox like this one is a good way to learn how to use this API safely.

Note that the NanoVG rendering will happen after the ImGui rendering, meaning that the NanoVG rendering will be on top of the ImGui rendering, regardless of the order in which the NanoVG and ImGui functions are called.

Most of the NanoVG API is bound to Lua, with the exception of the following functions:

-
-
-
-
-

In addition, the enums and some constructors for the structures used in NanoVG are available as extra values and functions. Please refer to the Lua source code for more details.

The general idea is that the emulator will call and before and after the Lua code is executed, and the Lua code will be able to call the other functions to draw the vector graphics.

The proper way to use the NanoVG API is to call , when in an ImGui window in order to queue the NanoVG rendering for this specific window.

The function takes a single argument, which is a function that will be called when the NanoVG rendering is being executed. The function will be called without argument.

All of the NanoVG functions are bound to the object, which is a proxy object to the proper NanoVG context, meaning it is only valid within the function passed to .

This allows the user to call the NanoVG functions without having to pass the NanoVG context as the first argument, as it is done automatically by the proxy object.

Note that the font used by the emulator is also loaded into the NanoVG context, meaning that it is possible to use without having to load a font first.

8.4.5 Example of using everything together

As the NanoVG rendering is very low level, and requires a viewport to draw to, it is required to use the ImGui API to draw some UI, grab the positions of the vector graphics

to add, and then queue some NanoVG calls within some ImGui context to draw the wanted vector graphics.

The following example will draw a red rectangle in the middle of the Output region. The rectangle will be 100x100 pixels in size, and will be drawn on top of the emulator rendering. It should follow around the Output region when resizing or moving the window.

In order to work, this example requires the code to be executed in the function of the Output shader invoker, so we can get the position of the Output region to draw to.

8.5 File API

8.5.1 Introduction & Rationale

While the normal Lua io API is loaded, there's a more powerful API that's more tightly integrated with the rest of the PCSX-Redux File handling code. It's an abstraction class that allows seamless manipulation of various objects using a common API.

The File objects have different properties depending on how they are created and their intention. But generally speaking, the following rules apply:

- Files are reference counted. They will be deleted when the reference count reaches zero. The Lua garbage collector will only decrease the reference count.
- Whenever possible, writes are deferred to an asynchronous thread, making writes return basically instantly. This speed up comes at the trade off of data integrity, which means writes aren't guaranteed to be flushed to the disk yet when the function returns. Data will always have integrity internally within PCSX-Redux however, and when exiting normally, all data will be flushed to the disk.
- Some File objects can be cached. When caching, reads and writes will be done transparently, and the cache will be used instead of the actual file. This will make reads return basically instantly too.
- The Read and Write APIs can haul LuaBuffer objects. These are Lua objects that can be used to read and write data to the file. You can construct one using the function. They can be cast to strings, and can be used as a table for reading and writing bytes off of it, in a 0-based fashion. The length operator will return the size of the buffer. The methods and are available. They also have a property that implicitly converts them to a Lua-Protobuf's , which can then be passed to .
- The Read and Write APIs can also function using Lua-Protobuf's buffers and slices respectively.
- If the file isn't closed when the file object is destroyed, it'll be closed then, but letting the garbage collector do the closing is not recommended. This is because the garbage collector will only run when the memory pressure is high enough, and the file handle will be held for a long time.
- When using streamed functions, unlike POSIX files handles, there's two distinct seeking pointers: one for reading and one for writing.

8.5.2 Common API for all File objects

All File objects have the following API attached to them as methods:

Closes and frees any associated resources. Better to call this manually than letting the garbage collector do it:

Reads from the File object and advances the read pointer accordingly. The return value depends on the variant used.

Reads from the File object at the specified position. No pointers are modified. The return value depends on the variant used, just like the non-At variants above.

Writes to the File object. The non-At variants will advances the write pointer accordingly. The At variants will not modify the write pointer, and simply write at the requested location. Returns the number of bytes written. The variants will in fact take any object that can be transformed to a string using .

Note that in this context, and refer to Lua-Protobuf's and objects respectively.

Some APIs may return a object, which is an opaque buffer coming from C++. The and methods can take a . It is possible to write a slice to a file in a zero-copy manner, which will be more efficient:

After which, the slice will be consumed and not reusable. The object is convertible to a string using , and also has two members: , which is a , and . Once consumed by the variants, the size of a slice will go down to zero.

Finally, it is possible to convert a object to a one using the function. However, the same caveats as for normal objects apply: it is fragile, and will be invalidated if the underlying Slice is moved or destroyed, so it is recommended to use it as a temporary object, such as an argument to . Still, it is a much faster alternative to calling which will make a copy of the underlying slice.

The following methods manipulate the read and write pointers. All of them return their corresponding pointer. The argument can be of the values , , and , and will default to .

These will query the corresponding File object.

If applicable, this will start caching the corresponding file in memory.

Same as above, but will suspend the current coroutine until the caching is done. Cannot be used with the main thread.

Duplicates the File object. This will re-open the file, and possibly duplicate all ressources associated with it.

Creates a read-only view of the file starting at the specified position, spanning the specified length. The view will be a new File object, and will be a view of the same underlying file. The default values of start and length are 0 and -1 respectively, which will effectively create a view of the entire file. The view may have less features than the underlying file, but will always be seekable, and keep its seeking position independent of the underlying file. The view will hold a reference to the underlying file.

In addition to the above methods, the File API has these helpers, that'll read or write binary values off their corresponding stream position for the non-At variants, or at the indicated position for the At variants. All the values will be read or stored in Little Endian, regardless of the host's endianness.

8.5.3 Creating File objects

The Lua VM can create File objects in different ways:

Basic files

The function will function on filesystem and network URLs, while the function will generate a memory-only File object that's fully readable, writable, and seekable. The argument of the function will determine what happens exactly. It's a string that can have the following values:

- : Opens the file for reading only. Will fail if the file does not exist. This is the default type.
- : Opens the file for reading and writing. If the file does not exist, it will be created. If it does exist, it will be truncated to 0 size.
- : Opens the file for reading and writing. If the file does not exist, it will be created. If it does exist, it will be left untouched.
- : Opens the file for reading and writing. Will fail if the file does not exist.
- : Opens the file for reading only. Will immediately start downloading the file from the network. The argument will be treated as a URL. The curl is the backend for this feature, and its url schemes are supported. The progress of the download can be monitored with the method.
- : As above, but suspends the current coroutine until the download is done. Cannot be used with the main thread.

Buffers

When calling with no argument, this will create an empty read-write buffer. When calling it with a cdata pointer and a size, this will have the following behavior, depending on type:

- (or no type): The memory passed as an argument will be copied first.
- : The memory passed as an argument will be referenced, and the lifespan of said memory needs to outlast the File object. The File object will be read-only.
- : It will acquire the pointer passed as an argument, and free it later using , meaning it needs to have been allocated using in the first place.

The constructor will return a sparse buffer that has a virtual 4GB span. It can be used to read and write data in the 4GB range, but will not actually allocate any memory until the data is actually written to. This is useful for doing operations that are similar to that of the PlayStation memory. The constructor will return a File object that's fully readable, writable, and seekable. Its size will always be 4GB. The returned object will have 3 additional methods:

- : Returns the lowest address that has been written to.
- : Returns the highest address that has been written to.
- : Returns the size of the buffer, which is the highest address minus the lowest address.

This is a useful object to use with the method, as it will allow you to create a view of a specific range of the 4GB memory. Specifically, will create a view of the entire memory that has been written to.

Network streams

The function will create a File object that will read from and write to the specified TCP address and port after connecting to it. The method will return true in case of a connection failure. The address is a string, and must be a strict IP address, no hostnames allowed. The port is a number between 1 and 65535 inclusive. As the name suggests, this object is a FIFO, meaning that incoming bytes will be consumed by any read operation. The method will return the number of bytes in the FIFO. Writes will be immediately sent over. There are no reception guarantees, as

the other side might have disconnected at any point. The method will return true when the opposite end of the stream has been disconnected and there's no more bytes in the FIFO. In addition to the normal API, a has a method called , which returns a boolean indicating the fifo is still connecting, meaning it's possible to verify if the fifo has successfully connected using the boolean expression .

Compressed streams

The function will create a read-only File object which decompresses the data from the specified File object. The argument is a File object, and the argument is an optional number that will be used to determine the size of the decompressed data. If not specified, the resulting file won't be seekable, and its method won't work, but the file will be readable until returns true. The argument is an optional string that needs to be equal to , and will determine whether the data is compressed using the raw deflate format, or the zlib format. Any other string means the zlib format will be used.

8.5.4 Iso files

There is some limited API for working with ISO files.

- will return an object representing the currently loaded ISO file by the emulator.
- will return an object opened from the specified argument, which can either be a filesystem path, or a object.

The following methods are available on the object:

The method has some magic built-in. The size argument is optional, and if missing, the code will attempt to guess the size of the underlying file within the Iso. It

will represent the size of the virtual file in bytes. The size guessing mechanism can only work on MODE2 FORM1 or FORM2 sectors, and will result in a failed File object otherwise. The mode argument is optional, and can be one of the following:

- : will attempt to guess the mode of the file. This is the default.
- : the returned File object will read 2352 bytes per sector.
- : the returned File object will read 2048 bytes per sector.
- : the returned File object will read 2336 bytes per sector. This can't be guessed. This is useful for extracting STR files that require the subheaders to be present.
- : the returned File object will read 2048 bytes per sector.
- : the returned File object will read 2324 bytes per sector.

The resulting File object will cache a single full sector in memory, meaning that small sequential reads won't read the same sector over and over from the disk.

The resulting File object will be writable, which will temporarily patch the CD-Rom image file in memory. It is possible to flush the patches to a PPF file by calling the method of the corresponding Iso object. When writing to one of these files, the filesystem metadata information will not be updated, meaning that the size of the file on the filesystem will not change, despite it being possible to write past the end of it and overflow on the next sectors. Note that while the virtual File object will enlarge to accommodate the writes, it will not be filled with zeroes as with typical filesystem operations, but instead will be filled with the existing data from the iso image. When applicable, sync headers, location, MODE2 subheaders will be added, and ECC and EDC will be recalculated on the fly, and the resulting data will be written to the virtual file, except for files opened in mode. The mode cannot be written to, and will throw an error if attempted.

The ISOReader object has the following methods:

This method is basically a helper over the method of the Iso object, and will automatically guess the mode and size of the file.

8.6 Webserver Lua API

When the webserver is enabled, it will expose the prefix, which can be used to execute Lua code on the emulator. When an endpoint with this prefix is called, the Lua table will be inspected to find a handler for the rest of the path in the endpoint. If a handler is found, it will be called with a request object representing the query, and it has to return a string, which will be sent back to the client as the response. If no handler is found, a 404 error will be returned. If an error occurs while executing the handler, a 500 error will be returned.

The request object has the following fields:

- is a table of the form data in the request. This is only available if the request is a POST request, and the content type is .
- is a table of the headers in the request.
- is the HTTP method of the request.
- is a table with more information about the URL. It has the following string fields:
-
-
-
-
-
-
-

If the returned string starts with the characters "HTTP/", then the web server will consider the response string is a full HTTP response with headers, and will send it as-is to the client. Otherwise, the response string will be sent as the body of a normal 200 response.

8.7 Memory and registers

8.7.1 FFI access

The Lua code can access the emulated memory and registers directly through some FFI bindings:

- will return a representing up to 8MB of emulated memory. This can be written to, but careful about the emulated i-cache in case code is being written to.
- will return a representing up to 512kB of the EXP1/Parallel port memory space. This can be written to.
- will return a representing up to 512kB of the BIOS memory space. This can be written to.
- will return a representing up to 1kB for the scratchpad memory space.
- will return a structured cdata representing all the registers present in the CPU:
- will return a representing the read LUT for the CPU.
- will return a representing the write LUT for the CPU.

8.7.2 Safer access

The above methods will return direct pointers into the emulated memory, so it's easy to crash the emulator if you're not careful. The method is safer, but will be slower:

- will return a object representing the full 4GB of accessible memory. All operations on this file will be translated to the emulated memory space. This is slower than the direct access methods, but safer. Any read or write operation will be clamped to the emulated memory space, and will not crash the emulator.

8.7.3 Memory mapping

PCSX-Redux will attempt to forward reads and writes for memory not mapped in the LUTs. This is useful for debugging, but will be slower than the direct access methods.

- will be called when a read is attempted to an unmapped memory address. The function should return an 8, 16, or 32-bit value to be returned to the CPU. - will be called when a write is attempted to an unmapped memory address. The function should return or indicating whether the write was handled.

8.8 Events

The Lua code can listen for events broadcasted from within the emulator. The following function is available to register a callback to be called when certain events happen:

Important: the return value of this function will be an object that represents the listener itself. If this object gets garbage collected, the corresponding listener will be removed. Thus it is important to store it somewhere that won't get garbage collected right away. The listener object has a method to stop the listener before its garbage collection time.

The callback function will be called from an unsecured environment, and it is advised to delegate anything complex or risky enough to .

The argument is a string that can have the following values:

- : The emulator is about to quit. The callback will be called with no arguments. This is where you'd need to close libuv objects held by Lua through luv in order to allow the emulator to quit gracefully. Otherwise you may soft lock the application where it'll wait for libuv objects to close.
- : A new ISO file has been mounted into the virtual CDRom drive. The callback will be called with no arguments.
- : The emulated GPU has just completed a vertical blanking interval. The callback will be called with no arguments.
- : The emulation execution has reached the beginning of the BIOS' shell. The callback will be called with no arguments. This is the moment where the kernel is properly set up and ready to execute any arbitrary binary. The emulator may use this event to side load binaries, or signal gdb that the kernel is ready.
- : The emulator resumed execution. The callback will be called with no arguments. This event will fire when calling , when the user presses Start, or other potential interactions.
- : The emulator paused execution. The callback will be called with a table that contains a boolean named , indicating if the pause is the result of an execution exception within the emulated CPU. This event will fire on breakpoints too, so if breakpoints have Lua callbacks attached on them, they will be executed too.
- : The emulator is resetting the emulated machine. The callback will be called with a table that contains a boolean named , indicating if the reset is a hard reset or a soft reset. This event will fire when calling , when the user presses Reset, or other potential interactions.
- : The emulator just loaded a savestate. The callback will be called with no arguments. This event will fire when calling , when the user loads a savestate, or other potential interactions. This is useful to listen to in case some internal state needs to be reset within the Lua logic.
- : The UI is being asked to move the assembly view cursor to the specified address. The callback will be called with a table that contains a number named , indicating the address to jump to.

- : The UI is being asked to move the memory view cursor to the specified address. The callback will be called with a table that contains a number named , indicating the address to jump to, and , indicating the number of bytes to highlight.
- : The emulator is dispatching keyboard events. The callback will be called with a table containing four numbers: , , , and . They are the same values as the glfw callback set by .
- : The emulator has updated the memory LUTs. The callback will be called with no arguments.

8.9 Breakpoints

If the debugger is activated, and while using the interpreter, the Lua code can insert powerful breakpoints using the following API:

Important: the return value of this function will be an object that represents the breakpoint itself. If this object gets garbage collected, the corresponding breakpoint will be removed. Thus it is important to store it somewhere that won't get garbage collected right away.

The only mandatory argument is , which will by default place an execution breakpoint at the corresponding address. The second argument is an enum which can be represented by one of the 3 following strings: , , , and will set the breakpoint type accordingly. The third argument is the width of the breakpoint, which indicates how many bytes should intersect from the base address with operations done by the emulated CPU in order to actually trigger the breakpoint. The fourth argument is a string that will be displayed in the logs about why the breakpoint triggered. It will also be displayed in the Breakpoint Debug UI. And the fifth and final argument is a Lua function that will be called whenever the breakpoint is triggered. By default, this will simply call . If the invoker returns , the breakpoint will be permanently removed, permitting temporary breakpoints for example. The signature of the invoker callback is:

The parameter will contain the address that triggered the breakpoint. For breakpoints, this is going to be the same as the current , but for and , it's going to be the actual accessed address. The parameter will contain the width of the access that triggered the breakpoint, which can be different from what the breakpoint is monitoring. And the parameter will contain a string describing the reason for the breakpoint; the latter may or may not be the same as what was passed to the function. Note that you don't need to strictly

adhere to the signature, and have zero, one, two, or three arguments for your invoker callback. The return value of the invoker callback is also optional.

For example, these two examples are well formed and perfectly valid:

The returned breakpoint object will have a few methods attached to it:

-
-
-
-

A removed breakpoint will no longer have any effect whatsoever, and none of its methods will do anything. Remember it is possible for the user to still manually remove a breakpoint from the UI.

Note that the breakpoint will run outside of any safe Lua environment, so it's possible to crash the emulator by doing something wrong which would normally be caught by the safe environment of the main thread. This is to ensure that the breakpoint can run as

fast as possible. In order to avoid this, it's possible to wrap the invoker callback in a `` call, which will catch any error and display it in the logs. For example:

This will ensure that the breakpoint will never crash the emulator, and will instead display the error in the logs, but it will also slow down the execution of the breakpoint. It's up to the user to decide whether or not this is acceptable.

It is safe to add or remove breakpoints from within a breakpoint callback, but it's not safe to remove the breakpoint that is currently being executed. For this specific case, simply return `` from the invoker callback, and the breakpoint will be removed after the callback returns.

8.10 Inline assembler

There is a Lua API for an inline MIPS assembler.

One can instantiate an assembler with , which will keep all the state of the assembler. The assembler can be used to assemble a string of MIPS code, and then compile it to memory or a file.

The object has the following methods:

- will parse the string and assemble it. It will return the assembler object itself, so it can be chained with the compile methods. The parser is fairly simple, but it should be enough for most cases. The parser should handle all of the basic MIPS instructions, all of the PS1's GTE opcodes, and many pseudo-instructions. It will also handle labels. The parser is more lenient than normal MIPS assemblers, and will accept some invalid syntax, but it will throw an error if it can't parse the code.
- will compile the assembled code to a table of values. This is useful for debugging, but not very useful for actually running the code. The is the address that the code will be loaded at, in order to handle relative jumps.
- will compile the assembled code to an indexable memory object, such as an ffi array. The memory object must be at least as large as the assembled code. The memory object will be modified in-place. The is the address that the code will be loaded at, in order to handle relative jumps. The is the address that the memory object starts at.
- will compile the assembled code to a file object. The file object must be at least as large as the assembled code. The file object will be modified in-place. The is the address that the code will be loaded at, in order to handle relative jumps. The is an optional argument which defaults to 0, and is the address that the file object starts at. Using a 0-based file address is relevant when using with the function, or when using a File object.

8.11 Handling of PSX binaries

There is some support for handling PSX binaries in the Lua API. The module has the following functions:

- : loads an input object into an output object. The input file must be a valid PSX binary, which can be in the formats CPE, PS-EXE, PSF, or ELF, and the output file must be at least 4GB large, which means it's really only suitable with the object, or the object returned by . The output file will be modified in-place. The output file will be loaded at the address specified in the binary header. If successful, the function will return an info structure with the following optional fields:
- : the entry point of the binary
- : the global pointer of the binary
- : the stack pointer of the binary
- : the region of the binary, which can be one of the following:
- : NTSC region
- : PAL region
- : compresses the input binary stream into a self-decompressing stream. The input must be a object, and the output must be a object. The is the address that the binary will be loaded at. The , , and are the entry point, global pointer, and

stack pointer of the binary. The is an optional table with the following optional fields:

- : the address that the compressed binary will be loaded at. If not specified, it will be set to a suitable address. Not specifying this will generate an in-place decompression binary, which doesn't require much extra memory. When specifying this, the whole output stream will be loaded at this specific address, and the decompression code will be located at its beginning, meaning both the entry point and the loading addresses will be the same.
- : the generated PS-EXE will not be padded to 2048 bytes. It will not be suitable to boot from cd-rom, as the BIOS requires binaries to be aligned to sector sizes, but many other tools like unirom+nops or caetla+catflap will be able to handle it properly.
- : a boolean specifying that the output stream will be suitable to boot as a PIO bytestream. Incompatible with or .
- : a boolean specifying that the produced binary will not try to call into the kernel, in case the kernel has been wiped out. Results in a slightly bigger binary, but is necessary when the retail kernel is not present.
- : a boolean specifying that the output stream will attempt to reboot the machine and load the binary, which can be useful when resetting the kernel.
- : a boolean specifying that the output stream will be a raw binary, without a PS-EXE header. The generated binary will be completely position independent, and will not require any special loading address. It is up to the user to ensure no overlap can happen by loading the file to a high enough address. This option can be used to generate embedded binaries within others, or to be loaded by other means, and executed by jumping to it. The option will be ignored when this is specified.
- : a boolean specifying that the output stream will be a ROM file suitable to be flashed on a cheat cartridge, as long as the cartridge itself has linear addressing, which is not necessarily the case for all cartridges. The option will be ignored when this is specified.
- : a boolean specifying that the output stream will be a CPE file, which is the file format used by the ancient toolchain by Sony. This can be useful when trying to load binaries with these ancient tools.
- : creates a PS-EXE binary from the input binary stream. The input must be a object, and the output must

be a object. The is the address that the binary will be loaded at. The , , and are the entry point, global pointer, and stack pointer of the binary.

The above methods can be used for example the following way:

Additionally, the module has the following functions:

- : compresses the input binary stream into a ucl-compressed stream. Both the input and output arguments must be objects. The output stream will be written at its current write pointer, and will be compressed using the UCL-NRV2E compression algorithm, which is a variant of the UCL compression algorithm. The output stream can be decompressed in-place with very little memory overhead. Simply place the compressed data at the end of the decompression buffer + 16 bytes. The stream doesn't require to be aligned in any particular way.
- : writes a MIPS UCL-NRV2E decompression routine to the output object, at its current write pointer. The function returns the number of bytes written, which at the moment is 340 bytes. The code is position independent, and has the following function signature:
-

8.12 Case studies

8.12.1 Spyro: Year of the Dragon

By looking up some of the gameshark codes for this game, we can determine the following memory addresses:

- is the number of lives.
- is the health of Spyro.
- is the number of unspent jewels available to the player.
- is the number of dragons Spyro released so far.

With this, we can build a small UI to visualize and manipulate these values in real time: